

Theseus Midterm Presentation Script

Roles

- **A:** Opening, progress, architecture, current implementation setup, challenges, AI usage, closing.
- **B:** Full live demo operation and demo explanation.

Target length: **10–12 minutes**

0:00–0:45 — Opening

Speaker: A

Good afternoon. We are Team Zisyphuz, and our project is **Theseus**.

Theseus is a weekly AI review system for goal-time alignment. The main idea is to help users compare what they planned, what they actually did, and what should be adjusted for the next week.

For the midterm, our focus is not the original proposal anymore. Our focus is the current implementation, the working local review path, and the live demo.

At this point, Sprint 1 and Sprint 2 are complete. The backend review engine and persistence path are working locally, and Sprint 3 frontend work is in progress through an active branch.

0:45–2:00 — Overall Project Progress

Speaker: A

On the progress slide, we are showing the original sprint timeline and our current status.

Sprint 0 was the foundation phase. We defined the project structure, basic review idea, and implementation direction.

Sprint 1 focused on backend persistence. We added the local data model and SQLite-backed storage path.

Sprint 2 focused on the rule-based review engine. This part is already implemented: the system can generate structured weekly review output from sample weekly data.

Sprint 3 is the frontend MVP. This is not final main yet. It is currently an active frontend branch, but it is already far enough to support the midterm demo.

So the honest current status is:

- Backend and review engine are ahead of the original plan.
- Frontend is in active implementation.
- Final integration, evaluation, and polish are still next.

2:00–3:20 — System Architecture

Speaker: A

This slide shows the system architecture.

The review pipeline is deterministic. It is not an autonomous AI agent, and it is not trying to replace a calendar.

The basic flow is:

Input → API → SQLite → Rules → Review

The input includes goals, weekly plans, and time logs.

The FastAPI backend validates and processes the data.

SQLite stores the local records, including goals, projects, time logs, and weekly reviews.

The review engine applies deterministic rules. These rules check evidence such as completed work, blocked items, drift, dormancy, and next-step signals.

Then the system creates structured review output, including wins, insights, risk flags, next steps, evidence, and generated text.

The important point is that the review is evidence-based. AI can help with wording and implementation support, but the system claim is still grounded in deterministic review logic.

3:20–4:10 — Current Implementation Overview

Speaker: A

Before the live demo, I want to separate the current implementation into two proof layers.

First, we have the **frontend demo branch**. This shows the app shell, the review screen, signals, tracking, and planning flow.

Second, we have the **backend and database proof**. This shows that the local API responds, the review script runs, and weekly review results can be persisted.

During the demo, B will walk through both layers.

One clarification: the frontend branch includes fallback demo data so the live demo remains stable if the backend is not running during presentation. The backend proof is still shown separately through the API health check and persisted review output.

4:10–8:40 — Live Demo

Speaker: B

Demo Part 1 — Frontend Branch

I will start with the frontend branch.

This is the active frontend branch, not the final merged main branch yet. We are using it to demonstrate the current app shell and user-facing flow.

The app has four main sections: review, signals, tracking, and planning.

Demo Part 2 — Review Screen

First, this is the review page.

The review page is the main weekly summary page. It shows the user's weekly state, wins, risks, and next step.

The goal is that the user can quickly understand what went well, what needs attention, and what should change next week.

For example, the review output can show whether backend progress is on track, whether deep-work blocks were protected, or whether a restart is needed for the next week.

This is the primary user-facing result of the review engine.

Demo Part 3 — Signals Screen

Next, I will show the signals page.

The signals page gives compact status indicators for the week. It includes plan, stage, goal, and energy status.

This page is not meant to be a full dashboard. It is meant to provide quick context before the user reads the weekly review.

The key idea is to reduce the review into a few visible signals, so the user can understand the weekly state faster.

Demo Part 4 — Tracking Screen

Now I will show the tracking flow.

This screen includes the focus-block timer interaction.

The timer is designed around time-blocked work. It supports the project goal by connecting planned focus time with actual work sessions.

This is important because Theseus is not just reviewing goals. It is trying to connect goals with actual time usage.

Demo Part 5 — Planning Screen

Finally, this is the planning screen.

The planning page gives a simple next-week cue, such as a restart block or a recommended morning block.

This connects the review result back into action.

So the frontend flow is:

Review → Signals → Track → Plan

That is the app-level demo.

Demo Part 6 — Backend Health Check

Now I will switch to backend proof.

First, I will show the local health endpoint.

```
GET /health
```

The response shows:

```
{  
  "status": "ok",  
  "service": "theseus-backend"  
}
```

This confirms that the local FastAPI backend is running.

Demo Part 7 — Persisted Review Script

Next, I will show the persisted review path.

We run the review script using sample weekly data.

```
python scripts/run_persisted_review.py
```

The script generates a weekly review and persists the result into SQLite.

The important table here is:

```
weekly_reviews
```

This proves that the backend review output is not only generated in memory. It can also be stored locally as a review record.

Demo Part 8 — Fallback Clarification

One important clarification: the frontend branch can use fallback demo data.

This is intentional for demo stability.

If the backend is not running during the live UI walkthrough, the app can still show the user flow. But the backend proof is demonstrated separately through the health endpoint and persisted review script.

So the current state is:

- Frontend demo branch is ready for presentation.
- Backend review and persistence work locally.
- Full integration and final evaluation are still next steps.

That completes the demo.

8:40–10:00 — Challenges and AI Usage

Speaker: A

After the demo, I want to explain our current risks and how we are controlling them.

The first risk is scope clarity.

The frontend is an active branch, not final main yet. So we should not present it as production-ready or fully merged.

The second risk is backend integration.

The frontend demo can use fallback data. This protects the demo flow, but we are careful to show backend proof separately through API and SQLite persistence.

The third risk is AI wording.

We are not claiming that an LLM is independently making final judgments. The review engine today is rule-based and deterministic.

AI was used as an implementation accelerator. It helped with scaffolding, reviewing API contracts, checking edge cases, writing scripts, and improving documentation.

But AI is not the source of truth.

The source of truth is the implemented code, local tests, API behavior, database persistence, and manual team verification.

So our boundary is:

AI assists. The team verifies.

10:00–10:45 — Closing

Speaker: A

To summarize, Theseus has moved from proposal scope into a working local review system.

At midterm, we have:

- Backend review logic working locally.
- SQLite persistence for weekly review output.
- A frontend demo branch showing the app flow.
- API and database proof for the implementation.
- Clear scope boundaries for what is complete and what is still in progress.

Our next steps are to finish frontend integration, collect feedback, evaluate the review quality, and prepare the final presentation version.

Thank you. We are ready for questions.

Optional Q&A Backup Answers

Q: Is the frontend fully connected to the backend?

Not fully yet. The frontend branch supports the current demo flow and includes fallback data for stability. Backend proof is shown separately through the local API health check and persisted review script.

Q: Is this using an LLM to generate the review?

Not in the core review engine today. The review engine is deterministic and rule-based. AI helped with implementation support, structure, and wording, but the review claims are verified through rules, scripts, and manual checks.

Q: What is already working?

The backend health endpoint works, the rule-based review engine works, weekly review output can be persisted into SQLite, and the frontend branch can demonstrate the current app flow.

Q: What is not finished yet?

Final frontend integration, evaluation, polish, and final presentation preparation are still in progress.

Q: Why use fallback data?

Fallback data keeps the live demo stable. It avoids the risk of a frontend demo failing because of a local backend connection issue. The backend is still proven separately through API and database output.